

C Minor Compiler

Software Requirements Specification

Version 1.0

9 October 2026

Document purpose

This specification establishes the functional, non-functional, interface, verification, and delivery requirements for a compiler for the course-defined C Minor language. It defines the compulsory implementation boundaries, approved external tools, target execution environment, and acceptance conditions.

Document identification

Field	Specification
Document title	C Minor Compiler — Software Requirements Specification
Document version	1.0
Source language	C Minor (course-defined subset; not complete ISO C23)
Reference target	Linux x86-64; System V AMD64 ABI; ELF
Compulsory student stages	Lexical analysis, syntax analysis, semantic analysis, intermediate-code generation, IR-to-backend lowering
Optional student stage	Machine-independent optimization
Permitted native backend	LLVM llc or QBE qbe
Acceptance basis	All COMPULSORY requirements, normative grammar, and verification criteria

Requirement classification

Requirements labeled COMPULSORY are mandatory for acceptance. Requirements labeled OPTIONAL define nonessential extensions. Requirement identifiers are stable references for implementation, tests, and traceability.

Contents

- 1 Scope and ownership
- 2 Approved external tools and prohibited substitutions
- 3 Compulsory C Minor language profile
- 4 Lexical analysis
- 5 Syntax analysis and AST
- 6 Semantic analysis and declarations
- 7 Expressions and evaluation
- 8 Statements and functions
- 9 Intermediate representation and optimization
- 10 Student lowering, approved backends and executable
- 11 Driver and diagnostics
- 12 Quality limits and testing
- 13 Deliverables and milestones
- 14 Optional extensions
- 15 Change and conflict control
 - Appendix A Normative compulsory grammar (EBNF)
 - Appendix B Acceptance examples (normative expected outcomes)
 - Appendix C Tool commands and backend examples
 - Appendix D Technical references

Interpretation rules

The requirements in Sections 1–15 and the normative appendices jointly define the compulsory baseline. Where interpretation is required, apply the explicit requirement, the normative grammar, the stated semantic rules, and the acceptance contract in that order. No full-C23 conformance claim is implied.

1 Scope and ownership

CM-SCP-001 COMPULSORY The compiler shall accept every compulsory C Minor program defined by this document, reject invalid/unsupported source with useful diagnostics, and preserve specified behavior for all defined compulsory programs.

CM-SCP-002 COMPULSORY Students shall personally implement lexical analysis, syntax analysis and AST construction, semantic analysis and symbol tables, a separately inspectable student IR, and the translation of that IR to an approved backend IL. Optimization by students is optional.

CM-SCP-003 COMPULSORY Input shall be exactly one self-contained .cm or .c file. No preprocessor is run or expected. Preprocessing directives and source line-splicing syntax are rejected rather than expanded or silently ignored.

CM-SCP-004 COMPULSORY The normal compiler command shall produce backend-generated x86-64 GNU-compatible assembly (.s); the team shall independently demonstrate assembly (.o), linking and execution by approved external tools.

CM-SCP-005 COMPULSORY The course language is C Minor, not ISO C23. C-like syntax and ordinary signed-int behavior follow the explicit teaching profile here; unsupported C23 constructs must not be presented as inherently invalid C23.

CM-SCP-006 COMPULSORY The student submission shall show a trace from source → tokens → AST → symbols/diagnostics → student IR → backend IL → assembly → object → executable, including at least one loop and one recursive or ordinary function call.

CM-SCP-007 COMPULSORY The baseline compiler shall be implementable using the approved publicly available dependencies identified in this specification. Student teams shall obtain and configure these dependencies and provide reproducible build and test procedures. A separate project-specific runtime, test harness, backend adapter, starter compiler, or distributed binary package is not a required dependency.

2 Approved external tools and prohibited substitutions

Role	Approved	Permitted boundary
Implementation languages	C, C++, Java, Python	Any one; ordinary libraries permitted.
Lexing	Handwritten or GNU Flex	Students author rules/actions/token policy.
Parsing	Handwritten, GNU Bison or ANTLR 4	Students author grammar/AST and error strategy.
Semantic analysis	Language standard libraries	Student-developed checks; no existing C frontend.
IR and lowering	Own code + non-semantic utility libraries	Students choose IR/control flow and emit backend IL.
Native backend	LLVM llc 23.1.3 OR QBE qbe 1.3	Backend translates supplied IL to assembly only.
Assembly/linking	GNU Binutils as, GCC driver and system linker	Only on generated .s/.o, not user C Minor source.
Build/testing	Make, CMake, Bash, Python, Git	Scripts owned and documented by team.

EXT-001 COMPULSORY The only default implementation languages are C, C++, Java and Python. Teams may use their standard libraries for maps, arrays, strings, files, diagnostics, processes and tests. A different implementation language requires an explicit common course-scope amendment.

EXT-002 COMPULSORY GNU Flex, GNU Bison and ANTLR 4, or handwritten equivalents, are permitted as parser/scanner mechanisms. Student-authored lexical specifications, grammar, AST actions and parse error handling are required. A complete third-party C parser or compiler frontend may not supply these stages.

EXT-003 COMPULSORY Students shall implement name/scope/type rules, student IR, control-flow lowering and backend-IL emission. Existing C-language semantic checkers, GCC/Clang frontends, C-to-LLVM translators, generators that accept original/regenerated C and emit LLVM/QBE IL, and prebuilt C Minor compilers are prohibited substitutes.

EXT-004 COMPULSORY Only LLVM llc and QBE qbe are approved machine-code-generating backend families. Teams choose exactly one for baseline acceptance. LLVM 23.1.3 and QBE 1.3 are reference releases; a different release of the same backend family is allowed only if the student documents its version, target and successful compatibility tests; baseline functionality remains unchanged.

EXT-005 COMPULSORY LLVM/QBE may perform native instruction selection, register allocation, machine stack-frame layout, ABI handling, assembly emission and internal optimizations. Their output is not student-developed optimization credit. An existing tool shall not substitute for student-developed source-language branch, call, return, scope or variable semantics.

EXT-006 COMPULSORY Public assembler, linker and host startup components may be used. GCC may be invoked with generated .s for assembling and .o for linking; GCC or Clang shall NEVER receive the original or regenerated C Minor program as C/C++ source for compilation.

EXT-007 COMPULSORY Preprocessing is prohibited in the C Minor source pipeline, including using gcc -E or clang -E to bypass source restrictions. Ordinary preprocessing performed while building the compiler implementation itself is allowed.

EXT-008 COMPULSORY Approved build and testing utilities may be used. Student teams shall develop and execute their own reproducible build and testing procedures and record tool commands, versions, observed results and expected results.

EXT-009 COMPULSORY Teams shall disclose third-party libraries, generated code, copied material and AI assistance according to the institution/course academic-integrity rules, and demonstrate ownership of all compulsory compiler logic during review.

EXT-010 COMPULSORY The compulsory C Minor language shall require no project-specific runtime library. Execution may rely on the standard startup and linking facilities of the installed Linux toolchain. A baseline C Minor program may call only functions defined within the same source file.

3 Compulsory C Minor language profile

Compulsory	Excluded from baseline
Signed 32-bit int locals, parameters, expression results; void return functions	char, bool, unsigned, float/double, pointers, arrays, structs, globals
ASCII identifiers; decimal integer literals 0..2147483647	Hex/octal/binary, floating, suffixes, strings/character literals
+ - * / %, comparisons, ! && , assignment =, function calls	++, --, compound assignments, bitwise, shifts, ?:, casts, comma operator
Blocks; if/else; while; for; break; continue; return; null statement	do-while, switch/case, goto, labels, advanced statements
Function prototypes, definitions, 0..6 int parameters, direct calls, recursion	Variadics, function pointers, separate translation units, external library calls
// and /* */ comments; ASCII whitespace; LF and CRLF	Preprocessing directives and backslash-newline splicing

CM-LANG-001 COMPULSORY The only value/object/parameter type is signed 32-bit int (two's-complement on the target). The keyword void is allowed only as a function return type or the sole marker (void) for an empty parameter list. There are no compulsory implicit conversions between different user value types.

CM-LANG-002 COMPULSORY A valid compilation unit contains one or more function declarations/definitions and exactly one definition int main(void), with no source definitions of global objects. Every other declared function must have exactly one compatible definition in the same file; unresolved declarations are source-profile errors.

CM-LANG-003 COMPULSORY Each function takes zero through six int parameters. Function names are declared before calls by an earlier definition or compatible prototype. Recursion and mutual recursion must work. Separate modules, external library declarations and built-in print functions are not part of the baseline.

CM-LANG-004 COMPULSORY Every declared local is one scalar int with optional initializer. A declaration declares exactly one name. Names may be reused in a nested inner block (shadowing), but not declared twice in the same scope. for-initializer declarations follow the scope of that for loop.

CM-LANG-005 COMPULSORY The only compulsory output from execution is the process exit status of main. Positive test programs shall return 0 for pass and a nonzero value for failure. Standard output, printf, cm_print_int and cm_print_char are neither compulsory nor predefined or reserved.

CM-LANG-006 COMPULSORY The compiler shall document exclusions and limits in --capabilities. The optional extension catalogue in Section 14 never changes the compulsory acceptance grammar.

4 Lexical analysis

CM-LEX-001 COMPULSORY Identifiers begin with an ASCII letter or underscore and continue with ASCII letters, digits or underscores. Track token source path, 1-based line and 1-based column. int, void, if, else, while, for, break, continue and return are the only supported keywords; other recognized C23 reserved words are not usable as identifiers.

CM-LEX-002 COMPULSORY Accepted integer literals are exactly 0 or a nonzero decimal digit followed by decimal digits, with an integer value at most 2147483647. Under this defined language profile, 00, 012, 0x10, 0b10, 4u, 1.0 and 2147483648 are unsupported/malformed literal forms, not multiple adjacent valid tokens. Unary minus is a separate operator; -2147483648 is outside this baseline literal rule.

CM-LEX-003 COMPULSORY Ignore ASCII whitespace (space, horizontal tab, vertical tab, form feed and LF/CRLF) and recognize both // comments through line end/EOF and non-nested /* ... */ comments. Preserve token boundaries: a/**/b yields identifiers a then b. Unterminated block comments are errors. Comments may contain # or %: without producing preprocessing diagnostics.

CM-LEX-004 COMPULSORY Recognize {};, and required operators + - * / % < <= > >= == != ! && || = using longest-token matching. Recognizable unsupported multi-character operators such as ++, --, +=, << or ? shall be reported as unsupported operators, not silently split into valid tokens.

CM-LEX-005 COMPULSORY At a preprocessing-directive position (first non-whitespace, non-comment item of a source line), # or %: shall raise UNSUPPORTED_PREPROCESSING_DIRECTIVE without invoking a preprocessor. The same characters elsewhere outside a comment are lexical errors; other directive syntaxes are never accepted. A # inside comments has no effect.

CM-LEX-006 COMPULSORY Source backslash immediately followed by LF or CRLF outside comments shall receive UNSUPPORTED_LINE_SPLICING. No line joining occurs. LF and CRLF shall produce correct source line/column locations. Unsupported non-ASCII source characters shall be diagnosed, not normalized.

CM-LEX-007 COMPULSORY Unknown characters, quoted string/character tokens, invalid literals and unterminated tokens shall yield controlled errors. The scanner shall never silently delete meaningful source characters. The lexer must support the full syntax specified in Appendix A, including constructs not illustrated by the acceptance examples.

5 Syntax analysis and AST

CM-SYN-001 COMPULSORY Implement every compulsory production in Appendix A and the listed precedence and associativity. Unsupported forms are rejected rather than treated as implied C23 support.

CM-SYN-002 COMPULSORY Construct an inspectable AST with function prototypes, function bodies, parameter declarations, locals, blocks, expressions, calls, conditionals, loops and jump/return statements.

CM-SYN-003 COMPULSORY Retain source spans in the AST. The --dump-ast command shall give stable readable output for a fixed input/implementation, sufficient to distinguish parenthesization and operator precedence.

CM-SYN-004 COMPULSORY A declaration may appear only as a block item or as a for initializer; a bare unbraced declaration where a statement is expected is not valid compulsory syntax. The else always belongs to the closest unmatched if.

CM-SYN-005 COMPULSORY Syntax recovery should report independent later errors when possible, but shall never accept invalid input or produce a successful final .s. Grammar rejection is distinct from a recognized unsupported language extension.

6 Semantic analysis and declarations

CM-SEM-001 COMPULSORY Implement file-level function signature lookup, function parameter scope, nested block scopes and for-initializer declaration scopes. Resolve each variable or function use to the correct declaration.

CM-SEM-002 COMPULSORY Enforce declaration-before-use, unique local/parameter declaration within a scope, legal inner shadowing, function prototype/definition compatibility and no undefined user function.

CM-SEM-003 COMPULSORY Annotate every AST expression with int or void result category and whether it is a modifiable int identifier lvalue. Reject undeclared variables, assignment to functions, assigning to a non-lvalue and using void results as values.

CM-SEM-004 COMPULSORY Check call argument counts and function categories; direct calls only, with 0..6 int arguments. Check loop-only break/continue; function return forms; the exact main signature; and the final-return course restriction in Section 8.

CM-SEM-005 COMPULSORY Report semantic errors before final backend emission. No complete .s file shall be produced from a source that fails syntax or semantic analysis.

CM-SEM-006 COMPULSORY Implement --dump-symbols with names, kinds, lexical scopes, source locations, function signatures and name-resolution relationships; outputs shall be reproducible for fixed input.

CM-SEM-007 COMPULSORY A compiler is not required to prove definite initialization, termination or absence of every undefined-behavior case. Compulsory runtime tests use programs within the defined behavior contract.

7 Expressions and evaluation semantics

CM-EXP-001 COMPULSORY Support decimal int literals, local/parameter identifiers, parenthesized expressions, direct named calls, unary + - !, binary + - * / %, relational < <= > >=, equality == !=, logical && || and assignment =.

CM-EXP-002 COMPULSORY Follow Appendix A precedence. Binary arithmetic/comparison/equality/logical operators associate left to right when chained. Assignment associates right to left. A syntactically allowed assignment expression is semantically valid only when its left operand denotes a modifiable local/parameter identifier (parentheses do not change that identity).

CM-EXP-003 COMPULSORY Every non-void expression evaluates to int. Comparisons, equality and logical expressions yield exactly 0 or 1. Conditions test whether int is zero, and !x yields 1 iff x is zero.

CM-EXP-004 COMPULSORY Signed integer division truncates toward zero, and remainder satisfies $a == (a/b)*b + a\%b$ for defined operands. Overflow, division by zero, INT_MIN divided by -1, and reading an uninitialized automatic int are undefined: runtime tests shall avoid these cases. No trapping/bounds diagnostics are required.

CM-EXP-005 COMPULSORY Logical && and || shall short-circuit. The right expression is not evaluated when the left determines the result, even if the right has function calls or assignment side effects.

CM-EXP-006 COMPULSORY For this course-defined C Minor profile, evaluation of binary operands and call arguments proceeds left to right, with each completed operand taking effect before the next starts; assignment evaluates its right operand before storing to its identifier target. This is an explicit teaching-language sequencing rule, not a claim that C23 imposes it generally.

CM-EXP-007 COMPULSORY Nested function calls, assignment expressions and expression statements must preserve the selected sequencing order through student IR and backend IL. Acceptance tests shall not assume behavior that this language profile leaves undefined.

CM-EXP-008 COMPULSORY All integer operations in lowered IR shall preserve signed 32-bit results for defined inputs. LLVM i32 and QBE w are approved representations; boolean tests in LLVM use appropriate i1 comparisons/branches while results exposed as C Minor int are normalized to i32 0 or 1.

8 Statements and functions

CM-STM-001 COMPULSORY Support blocks, mixed local declarations/statements, expression statements, empty statements, if/else, while, for, break, continue and return.

CM-STM-002 COMPULSORY for initializer may be omitted, be an expression or be one local int declaration; condition and update may independently be absent. Missing condition means true. A for-declared variable lives through the loop body/update and is invisible afterward.

CM-STM-003 COMPULSORY break exits the innermost enclosing loop; continue in while goes to condition recheck, while continue in for goes to the update expression then condition recheck. Outside a loop, either is invalid.

CM-STM-004 COMPULSORY Support int- and void-returning functions, up to six int arguments, multiple early returns, forward prototypes and direct/mutual recursion. main must be exactly int main(void).

CM-STM-005 COMPULSORY In an int-returning function, return must supply an int expression. In void functions, return; has no expression and reaching the closing brace is allowed. Return-type mismatches are errors.

CM-STM-006 COMPULSORY To avoid a general control-flow return-path proof, every int function, including main, shall have as its last top-level block item an explicit return <int-expression>; statement. Other earlier returns are permitted. This is a stated C Minor course restriction, not the full C23 rule.

9 Student IR and optional optimization

CM-IR-001 COMPULSORY Define a documented, inspectable student intermediate representation (e.g., three-address code or control-flow graph) distinct from both source AST and selected backend IL. Generating backend IL directly from AST without an independent student IR does not satisfy this stage.

CM-IR-002 COMPULSORY The IR shall represent constants, unique local/parameter identities, assignment, signed arithmetic, comparison, boolean normalization, branches, labels, direct calls, call arguments, return values and function boundaries.

CM-IR-003 COMPULSORY Explicit IR control flow shall preserve if/else, loops, break/continue, early returns, recursion and short-circuit semantics; label destinations and temporary definitions shall be consistent.

CM-IR-004 COMPULSORY Provide a deterministic `--dump-ir` output. The student IR dump shall not be a renamed or verbatim copy of LLVM/QBE IL; reviewers must be able to identify the meaningful student-developed intermediate form and the translation to backend IL.

OPT-IR-001 OPTIONAL Student-developed constant folding, constant propagation, dead-code elimination and similar transformations are optional. Teams claiming them shall supply before/after IR and regression evidence; backend-internal optimization receives no student optimization credit.

10 Student lowering, approved backends and executable

CM-CG-001 COMPULSORY Students shall write the transformation from student IR to textual LLVM IR (.ll) or QBE IL (.ssa); generated backend IL shall be syntactically and semantically valid for the selected released backend. The backend may handle machine instruction selection, register allocation and ABI encoding.

CM-CG-002 COMPULSORY Students shall translate variable identity, reads/writes, branch conditions, expression temporaries, calls, function entry/exit and returns. Backend IL may use abstract storage operations (LLVM `alloca/load/store` or QBE `alloc4/loadw/storew`) or correctly formed SSA/phi values. Students are NOT required to compute physical addresses, stack-frame offsets, processor register assignments, or write assembly by hand.

CM-CG-003 COMPULSORY LLVM mode shall use signed i32 arithmetic, i32 parameter/return values, i1 branch tests, and explicit int-valued normalization for comparison/logical results. QBE mode shall use w (32-bit word) values, signed arithmetic where required, and valid QBE control-flow instructions. All selected IL must respect C Minor semantics.

CM-CG-004 COMPULSORY The selected backend must output .s from student-generated backend IL. The students must use existing GNU-compatible assembler and linker facilities on this .s to generate .o and an executable that runs correctly. Direct backend object output without the .s artifact does not satisfy the baseline.

CM-CG-005 COMPULSORY The team must identify its selected backend in `--capabilities` and may implement both only as an optional extension. The default CLI uses the team-selected backend; `--backend=<name>` may select only a backend that the implementation actually supports.

CM-CG-006 COMPULSORY The default baseline flags are `LLVM llc -O0 -mtriple=x86_64-pc-linux-gnu -filetype=asm` or `QBE qbe -t amd64_sysv`. QBE has no required matching "optimization off" switch; QBE internal optimization remains permitted and is not student optimization.

CM-CG-007 COMPULSORY Any backend error (including unavailable backend executable, invalid backend IL, failure to emit assembly) shall propagate as failure. New final .s shall be replaced atomically only after successful backend completion. Existing destination .s must survive a failed replacement; temporary artifacts must be cleaned.

CM-CG-008 COMPULSORY Executable construction shall use ordinary Linux executable startup and linking facilities available with the selected system toolchain (for example, the GCC linker driver). The function `main` shall be exported with the specified `int main(void)` interface and ABI. No additional project-specific runtime interface is required.

CM-CG-009 COMPULSORY The compiler shall implement any helper code needed for its student-owned IR-to-backend lowering. Ordinary formatting and data-structure utilities may be reused, but third-party packages performing source-language semantic translation or complete student IR-to-LLVM/QBE lowering are prohibited substitutes.

11 Compiler driver and diagnostics

CM-CLI-001 COMPULSORY Ship a runnable command named `cminorcc` (binary or executable wrapper). It accepts exactly one .cm or .c source file in default mode. On success, `cminorcc input.cm -o out.s` generates GNU-compatible assembly; without `-o` the default path is `input.s`.

CM-CLI-002 COMPULSORY Implement `--check`, `--dump-tokens`, `--dump-ast`, `--dump-symbols`, `--dump-ir`, `--emit-backend-ir`, `--capabilities`, `--help`, `--version`, and `--backend=llvm|qbe` (subject to actual supported choice). `--check` runs through semantic analysis and does not invoke the native backend.

CM-CLI-003 COMPULSORY Each `--dump-*` action prints a deterministic human-readable stage dump to stdout and does not create final `.s`; `--emit-backend-ir` writes textual `.ll` or `.ssa` to `-o PATH` when supplied, or to stdout otherwise. Options `--check` and stage dumps are mutually exclusive with each other for simplicity; incompatible combinations cause `command-misuse` status.

CM-CLI-004 COMPULSORY `--capabilities` reports language version, target, supported backend(s) and exact installed version(s), accepted source forms, resource limits, disabled preprocessing, and optional extensions. A selectable but unimplemented backend shall not be listed as supported.

CM-CLI-005 COMPULSORY Exit codes: 0 on successful requested action; 1 on source, file, unsupported-feature, compiler or external-backend error; 2 on invalid command line. Diagnostics go to stderr; stdout holds dumps or requested textual backend IL. A failed compilation must never be reported as success.

CM-DIA-001 COMPULSORY Source errors contain a stable category/code, severity, file path, 1-based line and column, and a concise explanation. Show related declaration location where helpful. External-tool errors identify the tool, command/arguments and its exit status; a source location is required only when available.

CM-DIA-002 COMPULSORY The compiler shall distinguish at least `LEXICAL_ERROR`, `SYNTAX_ERROR`, `NAME_ERROR`, `TYPE_ERROR`, `CALL_ERROR`, `RETURN_ERROR`, `CONTROL_FLOW_ERROR`, `UNSUPPORTED_FEATURE`, `UNSUPPORTED_PREPROCESSING_DIRECTIVE`, `UNSUPPORTED_LINE_SPLICING`, `IO_ERROR`, `BACKEND_ERROR` and `COMMAND_ERROR` (or documented equivalents).

CM-DIA-003 COMPULSORY The scanner shall not preprocess `#include`, `#define`, `#if`, `#ifdef`, `#pragma`, `#embed` or `%`: directive forms. Comments containing these spellings remain valid. Unsupported C23 features shall be reported as outside C Minor when recognizable.

CM-DIA-004 COMPULSORY Parser recovery may report independent errors but must not cause repeated or unbounded cascades, a crash, a hang, or successful assembly from invalid source. The compiler shall have a controlled response to missing input, permission failures, resource exhaustion and backend failure.

12 Quality minima and verification

Dimension	Baseline minimum
Input size	32 KiB valid input; 4,000 tokens
Nesting	16 levels of block nesting and 16 expression nesting levels
Definitions	32 user-defined functions and 6 parameters per function
Locals	64 local declarations within one block
Line length	2,048 source characters
Test cases	At least 50 distinct positive/negative cases, with production and boundary coverage
Robustness	100 mutated or malformed inputs; per-case compiler timeout 5 s
Environment	Linux x86-64, System V AMD64 ABI; offline after dependencies installed

CM-NFR-001 COMPULSORY The compiler source and test workflow shall build/run offline after installation of documented public dependencies in Linux x86-64. Students may develop elsewhere (including WSL2), but must demonstrate the required Linux target.

CM-NFR-002 COMPULSORY All minima in the preceding table are compulsory. Source exceeding a documented declared implementation limit may be rejected with a controlled `LIMIT_ERROR`; every program inside the minimum supported limits must work.

CM-NFR-003 COMPULSORY Students shall ensure no unexplained crash, hang or compiler memory corruption on required tests. Every one of at least 100 distinct malformed/mutated inputs has a five-second compiler timeout; unknown outcomes are not passes.

CM-NFR-004 COMPULSORY For identical source, source-relative paths, compiler version, backend version, target and options, the stage dumps, student IR and emitted backend IL shall be reproducible. Deterministic .s is expected for the same complete installed toolchain and configuration.

CM-NFR-005 COMPULSORY When the driver launches external programs, it must use structured process arguments or equivalently safe quoting, not source-derived shell command interpolation. Source file names must never be evaluated as shell commands.

CM-VER-001 COMPULSORY Each compulsory requirement ID shall have a traceability record with one or more tests/inspection checks, expected result, observed result and evidence. No untested compulsory requirement can be marked complete.

CM-VER-002 COMPULSORY At least 50 distinct positive/negative cases shall cover all compulsory grammar constructs, precedence, comments, rejected directives, scopes, function signatures, control flow, short-circuiting, calls, recursion, int behavior, diagnostics and output preservation.

CM-VER-003 COMPULSORY Execution tests shall run student source through student lexer/parser/semantics/IR/lowering, selected backend, existing assembler and linker, then compare main exit status with reviewed expected results. Tests must prove that no original/regenerated C source is compiled by GCC/Clang.

CM-VER-004 COMPULSORY Students shall submit reproducible build/compile/assemble/link/test commands and record tool versions and exit statuses. Use of the installed system startup/runtime is permitted; no course-provided runtime functions or harness may be presumed.

CM-VER-005 COMPULSORY Compulsory execution cases shall avoid undefined operations (signed overflow, divide by zero, INT_MIN/-1, uninitialized reads). Negative source tests shall verify nonzero failure statuses and meaningful error locations. Optional failures and missing extensions do not invalidate a fully passing baseline.

CM-VER-006 COMPULSORY Successful baseline acceptance requires clean build; all compulsory requirements and tests passed; assembly, object, executable and source-to-output evidence; documentation; and a student-owned implementation walkthrough.

13 Deliverables and milestones

CM-DEL-001 COMPULSORY Submit all student compiler source, runnable cminorcc command, dependency list (versions and download sources), build steps, test suite and commands, representative input programs, full reproducibility instructions and results.

CM-DEL-002 COMPULSORY Submit a user guide defining accepted grammar and exclusions, driver flags, expected exit codes, reference environment, selected backend, limits and any optional extensions.

CM-DEL-003 COMPULSORY Submit a developer report explaining lexer/token model, grammar/AST, scopes and semantic checks, student IR design, conversion to backend IL, native backend integration, error strategy, and major implementation decisions with code pointers.

CM-DEL-004 COMPULSORY Submit output evidence: tokens, AST, symbols, student IR, lowered LLVM/QBE IL, .s, .o and executable generation. Include at least one trace covering a loop and a function call and representative failure diagnostics.

CM-DEL-005 COMPULSORY Submit test results and requirement traceability ledger; list reused packages, licenses, and assistance under the applicable academic-integrity policy. Every student team must be able to explain its compulsory components.

CM-DEL-006 COMPULSORY The project submission shall be self-contained with respect to student-developed source, tests, documentation, and reproducibility instructions, apart from the publicly obtainable tools and system components authorized by Section 2. Team size, assessment weights, deadlines and submission procedures are determined separately by course administration.

Milestone	Student-owned exit evidence
M1: environment/scope	Approved tool installation and versions, target validation, compiling a tiny backend-IL sample from Appendix C.
M2: lexer/parser	Tokens, AST, grammar coverage, unsupported directive/comment diagnostics.
M3: semantics	Symbol tables, scope/type/call/return checks and diagnostics.
M4: student IR	Separate readable IR for conditions, loops, calls, short-circuit.

Milestone	Student-owned exit evidence
M5: lowering/integration	LLVM/QBE IL → .s → .o → executable, correct process return.
M6: release	Tests, documentation, traceability, student walkthrough; optional optimizer separately.

14 Optional extensions

OPT-EXT-001 **OPTIONAL** Additional scalar types (char, bool, floating), arrays, global objects, pointers or aggregates may be implemented only with student-developed supporting semantics and backend lowering, documented dependencies, and corresponding tests.

OPT-EXT-002 **OPTIONAL** Extra operators/literals, do-while, switch or advanced declarations may be added only as clearly identified tested extensions. Baseline programs and exclusions must continue to behave correctly.

OPT-EXT-003 **OPTIONAL** Students may add another approved backend only after their selected backend passes the complete compulsory suite. No additional machine-code backend family is preapproved.

OPT-EXT-004 **OPTIONAL** Student-owned optimization passes and optional output/reporting mechanisms may be included, with proof of preserved defined behavior. Backend internal optimization is never student optimizer credit.

OPT-EXT-005 **OPTIONAL** A team may develop its own assembler/linker/runtime or add external library calls as a separately documented extension, but the compulsory project shall never require them. Added C Minor input preprocessing remains excluded by the baseline project policy.

15 Change and conflict control

CM-CHG-001 **COMPULSORY** This Version 1.0 establishes the compulsory baseline. Any change to compulsory requirements, the accepted grammar, approved tool families, reference target, or acceptance criteria shall be issued through a documented course-wide amendment before the affected work is evaluated.

CM-CHG-002 **COMPULSORY** Optional features must preserve baseline mode; --capabilities and reports identify scope, dependencies and test status. An optional feature cannot replace or excuse a missing compulsory feature.

CM-CHG-003 **COMPULSORY** Each corrected compulsory defect requires an automated regression case; silent substitution of previously excluded behavior is not permitted.

Appendix A Normative compulsory grammar (EBNF)

EBNF: quoted text = literal token; {X} = zero or more; [X] = optional; | = alternatives. Whitespace/comments can separate tokens but cannot change token identities. Lexical and semantic conditions in Sections 3–8 remain normative.

```

program           = { external } ;
external          = function_decl | function_def ;
return_type       = "int" | "void" ;
function_decl     = return_type identifier "(" prototype_params ")" ";" ;
function_def      = return_type identifier "(" definition_params ")" block ;
prototype_params  = "void" | prototype_param { "," prototype_param } ;
prototype_param   = "int" [ identifier ] ;
definition_params = "void" | definition_param { "," definition_param } ;
definition_param  = "int" identifier ;
local_decl       = "int" identifier [ "=" assignment_expr ] ";" ;
block            = "{" { block_item } "}" ;
block_item       = local_decl | statement ;
statement        = block | expr_stmt | if_stmt | while_stmt | for_stmt
                  | "break" ";" | "continue" ";"
                  | "return" [ assignment_expr ] ";" ;
expr_stmt        = [ assignment_expr ] ";" ;
if_stmt          = "if" "(" assignment_expr ")" statement
                  [ "else" statement ] ;
while_stmt       = "while" "(" assignment_expr ")" statement ;
for_stmt         = "for" "(" for_init [ assignment_expr ] ";"
                  [ assignment_expr ] ")" statement ;
for_init         = local_decl | [ assignment_expr ] ";" ;

```

A.2 Expressions and lexical forms

```

assignment_expr  = logical_or [ "=" assignment_expr ] ;
logical_or       = logical_and { "||" logical_and } ;
logical_and      = equality { "&&" equality } ;
equality         = relational { ( "==" | "!=" ) relational } ;
relational       = additive { ( "<" | "<=" | ">" | ">=" ) additive } ;
additive         = multiplicative { ( "+" | "-" ) multiplicative } ;
multiplicative   = unary { ( "*" | "/" | "%" ) unary } ;
unary            = ( "+" | "-" | "!" ) unary | primary ;
primary          = integer_literal | identifier [ "(" [ arg_list ] ")" ]
                  | "(" assignment_expr ")" ;
arg_list         = assignment_expr { "," assignment_expr } ;
identifier       = (ASCII letter | "_" ) {ASCII letter | digit | "_" } ;
integer_literal  = "0" | ("1" ... "9") {"0" ... "9"} ;

```

Semantic guards: the left side of assignment denotes an identifier lvalue (optionally parenthesized); functions must be visible and have compatible definitions; no global declarations; exactly one `int main(void)`; 0–6 parameters; every int function ends with an explicit top-level return expression. The grammar intentionally permits syntactically shaped but semantically invalid assignments so the semantic analyzer can reject them.

The ordinary C23 spelling `f()` is not part of this teaching grammar: functions with zero arguments use `f(void)` for declarations/definitions and `f()` when CALLING. Calls follow the identifier (`arg_list`) branch above. No comma operator is included.

Appendix B Acceptance examples (normative expected outcomes)

B.1 Positive: precedence, comments and local variables

```

int main(void) {
    // A directive-looking #include is just a comment.
    int a = 10; /* a non-nested block comment */
    int/**/b = 2;
    int total = a + b * 3;
    if (total != 16) return 1;
    return 0;
}

```

Expected process exit status: 0. Checks comment handling, token boundaries, precedence and integer expressions.

B.2 Positive: prototypes and recursion

```

int fact(int n);
int main(void) {
    if (fact(5) != 120) return 1;
    return 0;
}
int fact(int n) {
    if (n <= 1) return 1;
    return n * fact(n - 1);
}

```

Expected process exit status: 0. Checks forward prototypes, visible calls, recursion and int returns.

B.3 Positive: loops, continue, shadowing and short circuit

```

int main(void) {
    int sum = 0;
    int touched = 0;
    for (int i = 1; i <= 5; i = i + 1) {
        if (i == 3) continue;
        { int sum = 9; sum = sum + 1; }
        sum = sum + i;
    }
    if (0 && (touched = 1)) return 1;
    if (1 || (touched = 2)) { ; }
    if (sum != 12) return 1;
    if (touched != 0) return 1;
    return 0;
}

```

Expected process exit status: 0. Checks $1 + 2 + 4 + 5 = 12$, for-loop continue target, independent shadowed local, and skipped short-circuit assignments.

B.4 Positive: function arguments and void functions

```
void noop(void) { return; }
int add(int a, int b) { return a + b; }
int main(void) {
    noop();
    if (add(19, 23) != 42) return 1;
    return 0;
}
```

Expected exit status: 0. Tests void-as-statement calls, int arguments and function results. The generated executable does not need a custom runtime.

B.5 Negative source and boundary cases

Example / situation	Expected diagnostic category
#include <stdio.h> at line start	UNSUPPORTED_PREPROCESSING_DIRECTIVE
#:define X 5	UNSUPPORTED_PREPROCESSING_DIRECTIVE
// #include <stdio.h>	Valid comment, no preprocessing error
int/**/name = 1; in a block	Valid separated tokens
int x /* unterminated	LEXICAL_ERROR
int x = 0x10;	UNSUPPORTED_FEATURE literal
int x = 2147483648;	UNSUPPORTED_FEATURE / literal range
int x = 012;	UNSUPPORTED_FEATURE / leading-zero literal
int x = 5; int x = 6; same scope	NAME_ERROR
int x[2];	UNSUPPORTED_FEATURE
float x; or int *p;	UNSUPPORTED_FEATURE
int f(void); no f definition	NAME_ERROR / undefined function
break; outside a loop	CONTROL_FLOW_ERROR
int main(void) { if (1) return 0; }	RETURN_ERROR: missing top-level final return
int main(void) { int x = 2; x++; return 0; }	UNSUPPORTED_FEATURE operator
int main(void) { return; }	RETURN_ERROR
void f(void) { return 1; }	RETURN_ERROR
Nonexistent backend executable	BACKEND_ERROR; preserve existing .s
File output directory unwritable	IO_ERROR; preserve existing .s

Negative snippets must be wrapped in an otherwise valid program when necessary. Expected diagnostic codes may be documented equivalents; location information must identify the real offending input. Positive output checks rely on main returning 0 rather than stdout.

Appendix C Self-contained tool recipes and backend examples

The following reference command sequences and minimal backend-IL samples define supported integration paths. The student compiler shall generate its own backend IL for submitted source programs.

C.1 LLVM backend path

```
# cminorcc is the student-developed program.
cminorcc --backend=llvm demo.cm --emit-backend-ir -o demo.ll
llc -O0 -mtriple=x86_64-pc-linux-gnu -filetype=asm demo.ll -o demo.s
```

```
gcc -c demo.s -o demo.o
gcc -no-pie demo.o -o demo
./demo
printf 'exit=%s\n' "$?"
```

Equivalent one-command student compile: `cminorcc --backend=llvm demo.cm -o demo.s` (the driver internally invokes `llc` on emitted LLVM IR). `-no-pie` simplifies this reference example; alternate compatible ELF linking configurations are permitted if tests pass.

C.2 Minimal LLVM IR smoke-test example

```
; For testing tool installation only; not credit as student compiler output.
target triple = "x86_64-pc-linux-gnu"
define i32 @main() {
entry:
    ret i32 0
}
```

C.3 QBE backend path

```
cminorcc --backend=qbe demo.cm --emit-backend-ir -o demo.ssa
qbe -t amd64_sysv -o demo.s demo.ssa
gcc -c demo.s -o demo.o
gcc -no-pie demo.o -o demo
./demo
printf 'exit=%s\n' "$?"
```

C.4 Minimal QBE IL smoke-test example

```
# For testing tool installation only; not credit as student compiler output.
export function w $main() {
@start
    ret 0
}
```

QBE uses `w` for a 32-bit word and `export` for the main symbol. LLVM `i32` and QBE `w` map to the compulsory source `int`. Tools must not receive the original C Minor file as C input. These examples specify the expected backend input forms and may be used to verify the local tool installation.

C.5 Driver behavior examples

```
cminorcc sample.cm -o sample.s           # normal compile to assembly
cminorcc --check sample.cm              # semantic validation only
cminorcc --dump-tokens sample.cm        # tokens on stdout
cminorcc --dump-ast sample.cm           # AST on stdout
cminorcc --dump-symbols sample.cm       # symbols on stdout
cminorcc --dump-ir sample.cm            # student IR on stdout
cminorcc --emit-backend-ir sample.cm -o sample.ll
cminorcc --capabilities                 # supported backend/profile
```

When QBE is selected, `--emit-backend-ir` shall use the `.ssa` output format instead of `.ll`. The normal `cminorcc` output remains `.s`, never a `.c` file compiled by GCC/Clang.

Appendix D Technical references

LLVM `llc` reference: <https://llvm.org/docs/CommandGuide/llc.html>

LLVM IR language reference: <https://llvm.org/docs/LangRef.html>

LLVM release 23.1.3: <https://discourse.llvm.org/t/llvm-23-1-3-released/92432>

QBE released versions (1.3 on 2026-06-01): <https://c9x.me/compile/releases.html>

QBE IL: <https://c9x.me/compile/doc/il-v1.2.html>

QBE home: <https://c9x.me/compile/>

GNU compiler driver: <https://gcc.gnu.org/onlinedocs/gcc/Overall-Options.html>

GNU Binutils assembler: <https://sourceware.org/binutils/docs/as/>

C23 published standard (background only): <https://www.iso.org/standard/82075.html>